

Project Part 1 — Design Specification (D1)

ScoreScript

A Domain-Specific Language for Student Grade Management

Student Number: 230104004090

Name: FATİH EMRE OĞAN

Submission Date: 8 May 2026

4.1 Language Overview

ScoreScript came to mind while I was working on this assignment. I thought: what if there was a language made for everyone involved in academic grading not just the instructor, but also teaching assistants who track quiz scores, and academic advisors who follow a student's progress across courses?

ScoreScript is a domain-specific language for academic grade management. With ScoreScript, you can define students with their id, name and grades, group them into classes, apply grade curves, compute averages, rank students and print transcripts. The syntax is simple and readable even someone with little programming experience can use it.

When designing ScoreScript, I prioritized readability and writability.

Sample Program:

```
student s1 {
  id: "230104004090",
  firstName: "Fatih Emre",
  lastName: "OGAN",
  grades: { math: 85, science: 90, history: 72 }
}

student s2 {
  id: "230104004089",
  firstName: "Omer Faruk",
  lastName: "OGAN",
  grades: { math: 91, science: 88, history: 95 }
}

group ClassZ23 = [s1, s2]
curve ClassZ23 by 5
average ClassZ23
rank ClassZ23
transcript s1
```

Language Evaluation:

ScoreScript prioritizes readability, writability, and reliability. Grade management is a repetitive and detail-sensitive task, so the language is designed to be concise and easy to use without requiring deep programming knowledge. Reliability is critical: a grading error directly affects a student's academic record, which is why ScoreScript enforces strict field ordering and rejects malformed programs with precise error messages. Performance and cost are knowingly sacrificed: a typical class has at most a few hundred students, so execution speed is simply not a concern for this domain.

4.2 Lexical Structure

The ScoreScript lexer recognises the following token categories. Each category is defined by a regular expression or explicit enumeration. The lexer processes tokens in the order listed; multi-character operators are matched greedily before single-character ones, and FLOAT literals are attempted before INT to prevent a value such as 60.0 from being tokenised as INT(60) followed by a dot and INT(0). Single-line comments beginning with `//` are stripped by the lexer and produce no tokens.

Keywords

Keywords are reserved identifiers. They are matched before the IDENT rule and cannot be used as variable or function names.

Category	Tokens
Statement / declaration	student group func for in if else return print by
Operations	curve average rank transcript
Type keywords	int float string bool void
Student field keywords	id firstName lastName grades
Boolean literals	true false

Identifiers

User-defined names for students, groups, variables, and functions follow the rule:

```
IDENT = [a-zA-Z][a-zA-Z0-9_]*
```

An IDENT that matches any keyword in the table above is classified as that keyword, not as IDENT.

Literals

Token	Pattern	Examples
INT	<code>[0-9]+</code>	0 85 100
FLOAT	<code>[0-9]+ "." [0-9]+</code>	60.0 3.14 99.5
STRING	<code>"\" { char } "\"</code>	"Fatih Emre" "OGAN"

The FLOAT rule is attempted before INT. An unterminated STRING (no closing double-quote before a newline or end of file) is a lexical error reported with the line number on which the literal opened.

Operators

Category	Tokens
Arithmetic	+ - * /
Unary	- !
Relational	< <= > >=
Equality	== !=
Logical	&&
Assignment / arrow	= ->

Multi-character operators (==, !=, <=, >=, ->, &&, ||) are recognised greedily. A single & or | not followed by its pair is a lexical error.

Separators

Token	Character
LBRACE / RBRACE	{ }
LPAREN / RPAREN	()
LBRACKET / RBRACKET	[]
COMMA	,
COLON	:

Comments

```
COMMENT = "//" { any character except newline }
```

ScoreScript supports single-line comments only. Block comments are not supported. This is a deliberate simplicity decision: the lexer remains single-pass with no nested state. Comments are consumed by the lexer and produce no tokens.

4.3 Syntax

The complete grammar of ScoreScript is given below in EBNF following the conventions of Sebesta §3.1–3.2. Terminal symbols appear in double quotes or as token names in ALL CAPS. The metasymbols { } denote zero or more repetitions; [] denote an optional element; | separates alternatives; parentheses group.

The grammar is unambiguous by construction. Every production is uniquely identified by its leading token, so the parser requires at most one token of lookahead (LL(1) property). The single disambiguation annotation required is the distinction between <var_decl> and <assign_stmt>, which is resolved by leading token: a declaration begins with a type keyword (int, float, string, bool), whereas an assignment begins with a bare IDENT. The dangling-else problem does not arise because all branch bodies are enclosed in mandatory braces.

Program Structure

```
<program>          ::= <decl_section> <stmt_section>

<decl_section>     ::= { <student_decl> | <group_decl> | <var_decl> |
<func_decl> }

<stmt_section>     ::= { <stmt> }
```

All declarations must precede all statements. This enforces declaration-before-use at the structural level: every name referenced in the statement section is guaranteed to be bound in the declaration section. A program that places a statement before the end of its declarations is rejected at parse time.

Declarations

```
<student_decl> ::= "student" IDENT "{  
    "id"      ":" STRING ","  
    "firstName" ":" STRING ","  
    "lastName" ":" STRING ","  
    "grades"  ":" "{" <grade_list> "}"  
    }"  
  
<grade_list> ::= <grade_entry> { "," <grade_entry> }  
<grade_entry> ::= IDENT ":" INT  
  
<group_decl> ::= "group" IDENT "=" "[" <ident_list> "]"  
<ident_list> ::= IDENT { "," IDENT }  
  
<var_decl> ::= <value_type> IDENT "=" <expr>  
  
<func_decl> ::= "func" IDENT "(" [ <param_list> ] ")" "->" <type>  
    "{" <func_body> "}"  
  
<param_list> ::= <param> { "," <param> }  
<param> ::= IDENT ":" <value_type>  
  
<func_body> ::= { <stmt> }
```

The `student_decl` rule encodes a fixed, mandatory field order: `id`, `firstName`, `lastName`, `grades`. The four field keywords are recognised as dedicated token types by the lexer, so any deviation from this order — a missing field, a reordered field, or an unknown field name — is caught at parse time with a precise error message.

Statements

```
<stmt> ::= <operation>  
    | <if_stmt>  
    | <for_stmt>  
    | <print_stmt>  
    | <return_stmt>  
    | <var_decl>  
    | <assign_stmt>  
    | <call_stmt>  
  
<operation> ::= "curve" IDENT "by" <expr>  
    | "average" IDENT  
    | "rank" IDENT  
    | "transcript" IDENT  
  
<if_stmt> ::= "if" <expr> "{" { <stmt> } }"  
    { "else if" <expr> "{" { <stmt> } }" }
```

```

[ "else" "{" { <stmt> } "}" ]

<for_stmt>      ::= "for" "student" IDENT "in" IDENT
                  "{" { <stmt> } "}"

<print_stmt>    ::= "print" <expr>
<return_stmt>   ::= "return" <expr>
<assign_stmt>   ::= IDENT "=" <expr>
<call_stmt>     ::= IDENT "(" [ <arg_list> ] ")"

```

Function bodies reuse the same <stmt> rule as the top-level statement section. This maximises writability: a programmer uses identical syntax whether inside a function or at the top level. The four domain operations (curve, average, rank, transcript) are each identified by a unique leading keyword, making the grammar LL(1) at the operation level.

The for loop requires the keyword student before the loop variable. This is a domain-specific design decision: iteration in ScoreScript is defined exclusively over groups of students. The mandatory keyword makes the domain intent explicit in the syntax and prevents the loop from being used for general-purpose iteration, which is outside the scope of the language.

The else if chain is assembled by the parser from two separate tokens (else and if) rather than a compound keyword. This avoids introducing a new token and does not cause ambiguity because all branch bodies are brace-delimited.

Types

```

<type>          ::= <value_type> | "void"
<value_type>    ::= "int" | "float" | "string" | "bool"

```

The void type is separated from value types at the grammar level. Parameters use <value_type>, not <type>, so a parameter declared void is a parse error. Variable declarations also use <value_type>. This pushes an entire class of type errors to parse time rather than type-check time.

Expressions

```

<expr>          ::= <or_expr>

<or_expr>       ::= <and_expr>          { "||" <and_expr> }
<and_expr>      ::= <equality_expr>    { "&&" <equality_expr> }
<equality_expr> ::= <relational_expr> { ("==" | "!=")
<relational_expr> }
<relational_expr> ::= <additive_expr> { ("<" | "<=" | ">" | ">=")
<additive_expr> }
<additive_expr> ::= <mult_expr>        { ("+" | "-") <mult_expr> }

```

```

<mult_expr>      ::= <unary_expr>      { ("*" | "/" ) <unary_expr> }

<unary_expr>     ::= "-" <unary_expr>
                  |  "!" <unary_expr>
                  |  <primary>

<primary>        ::= INT
                  |  FLOAT
                  |  STRING
                  |  "true" | "false"
                  |  IDENT
                  |  <func_call_expr>
                  |  "(" <expr> ")"

<func_call_expr> ::= IDENT "(" [ <arg_list> ] ")"
<arg_list>       ::= <expr> { "," <expr> }

```

Operator precedence is encoded structurally in the grammar: each layer only recurses into the layer immediately above it in tightness. No external precedence table or annotation is required. The seven precedence levels, from tightest to loosest binding, are:

Level	Grammar Rule	Operator(s)	Associativity
1 (tightest)	<unary_expr>	- !	Right
2	<mult_expr>	* /	Left
3	<additive_expr>	+ -	Left
4	<relational_expr>	< <= > >=	Left
5	<equality_expr>	== !=	Left
6	<and_expr>	&&	Left
7 (loosest)	<or_expr>		Left

All binary operators are left-associative, encoded by the repetition form { op <next_layer> } rather than right-recursion. Unary operators are right-associative by the recursive definition of <unary_expr>, permitting double negation (--x) and double logical negation (!!b).

Conditions in if and for statements are plain <expr> nodes. The parser accepts any expression as a condition; the type checker is responsible for rejecting conditions that do not evaluate to bool. This separates syntactic and semantic concerns as described in Sebesta §6.12.

4.6 Names, Binding, Scope, and Lifetime

Legal Identifiers

In ScoreScript, an identifier must start with a letter and can be followed by letters, digits, or underscores. So `s1`, `ClassZ23`, and `letter_grade` are valid, but `2fast` and `_temp` are not. Reserved keywords like `if`, `student`, and `curve` cannot be used as identifiers.

Binding Time:

Compile time:

Variable types, function signatures, and student/group names are bound at compile time. For example, when you write `int x = 5`, the type of `x` is fixed as `int`. Function parameter types and return types are also determined at compile time.

Runtime:

Variable values are bound at runtime the actual value of `x` is assigned when the program executes. Student field values such as grades are also read at runtime. The for-loop iterator variable is re-bound at each iteration during execution.

Scoping:

ScoreScript uses static scoping. This means a variable's scope is determined by where it is written in the code. If we used dynamic scoping, the same function could give different results depending on which function called it. For example, if two functions both have a variable named `avg`, they could interfere with each other. This would make grade calculations unpredictable, which is exactly what we want to avoid in a grading language.

Lifetime:

In ScoreScript, variables have different lifetimes depending on where they are declared. Top-level declarations such as `students`, `groups`, and global variables have static lifetime, they are created when the program starts and exist until the program ends. Local variables declared inside a function body have stack-dynamic lifetime they are created when the function is called and destroyed when the function returns. The for-loop variable also has stack-dynamic lifetime it is created at the start of the loop body and destroyed when the loop ends. It is not visible outside the loop. ScoreScript has no explicit-heap-dynamic variables. There is no manual memory allocation or deallocation. This is a deliberate decision it keeps the language simple and eliminates a whole class of runtime errors like memory leaks, which supports the reliability criterion.